

Failure–Recovery Calculus for World-Computer Virtual Machines: Monadic Laws, Seven Implications, and the EVM as a Privileged Experimental Substrate

Abstract

This article develops a Failure–Recovery (FR) theory for permissionless world-computer virtual machines (VMs) by treating “composability” not as the existence of successful execution paths, but as the existence of recovery-preserving morphisms that compose across contract interactions. The central move is to replace informal notions of “risk” with a typed and observable *failure surface* and to require *recovery closure*—the absence of unhandled failure residues at designated boundaries. We introduce an FR category whose morphisms are effectful computations that return either a value or a classified failure together with evidence, and whose composition is defined by recovery-aware Kleisli composition. We then motivate and adopt monadic laws as semantic invariants ensuring that recovery structure is stable under composition. From this foundation we derive seven non-trivial implications, including a redefinition of composability, a separation of safety from composability, a reframing of oracle problems as morphism-existence problems, and a formal account of “human-in-the-loop” intervention as categorical extension at morphism gaps. Finally, we explain why the Ethereum Virtual Machine (EVM) forms a privileged experimental substrate for FR: its synchronous call semantics, transaction-level atomicity via revert, and strong traceability yield a canonical sequential operational semantics suitable for evidence-carrying recovery reasoning at scale.

Index Terms— blockchain virtual machines, composability, monads, algebraic effects, smart contracts, failure recovery, operational semantics.

I. Introduction

“World computer” systems aim to provide a public, permissionless execution substrate for programs that interact and compose. In practice, the limiting factor is rarely the existence of “successful” paths, but rather the behavior of programs under adverse conditions: contention, partial information, adversarial ordering, external dependencies, and governance or oracle failures. This motivates a semantic shift:

- **Classify failures instead of hiding them** (avoid “Unknown” failures).
- **Force observability** (ensure failures do not progress silently).
- **Localize blast radius** (confine failures to explicit boundaries).
- **Require recovery procedures** (including human annotation where necessary).

The present work formalizes these principles as an FR theory. The key claim is:

Composability is the existence of recovery-preserving composition, not the abundance of success paths.

Technically, we define a category (or, more precisely, a family of categories indexed by boundary/capability regimes) whose morphisms are computations returning either a value or a classified failure plus evidence. We then organize these morphisms monadically (Kleisli category) and interpret monadic laws as invariants guaranteeing that recovery structure is stable under composition. This structure yields seven implications and clarifies why the EVM is unusually suitable for FR experimentation.

II. Background and Motivation

A. EVM as a sequential, synchronous, revert-atomic substrate

The EVM specifies synchronous inter-contract calls (e.g., `CALL`, `DELEGATECALL`) and transaction-level atomicity: execution either completes, or state changes are reverted on failure. The Yellow Paper provides the formal reference for EVM state transition semantics and execution model. ([ethereum.github.io](<http://ethereum.github.io>))[1])

B. Algebraic effects and handlers as a design pattern for “failure as data”

The FR approach aligns with a broad PL principle: model effects as algebraic operations and interpret them with handlers. This yields modular semantics and systematic recovery. ([homepages.inf.ed.ac.uk](<http://homepages.inf.ed.ac.uk>))[2])

C. Contrast: causally ordered, object-centric execution

Some modern systems intentionally avoid total ordering for many transactions and instead rely on causal ordering and object ownership regimes; e.g., Sui can bypass consensus for “owned-object” transactions while requiring consensus/ordering for “shared-object” transactions. ([Sui Documentation])[3])

This contrast is important for §VI: it explains why EVM-like FR reasoning is unusually direct in EVM but becomes inherently multi-track (owned/shared paths) in object-centric designs.

III. Preliminaries: Sets, Boundaries, and Evidence

We work in ordinary set-theoretic mathematics; “Bourbaki-level” rigor is approximated by explicit sets, maps, and closure properties, but we do not fully axiomatize ZF here.

A. States and boundaries

Let $\mathcal{X}$ be the set of global VM states. A *boundary* is an abstract regime that constrains and interprets computations (capabilities, epochs, storage regions, call contexts). We model boundaries as elements of a set $\mathcal{B}$.

Intuitively:

- $(b \in \mathcal{B})$ captures what must be localized (e.g., “this storage region”), what must be observable (e.g., “emit evidence”), and what recovery obligations exist (e.g., “must produce a resolution plan”).

B. Failure surface, evidence, and recovery closure

We fix the following sets:

- **Failure modes** (failure surface): $\mathcal{F}$
Elements $(f \in \mathcal{F})$ are *classified* failures (finite sum types in implementations).
- **Evidence**: $\mathcal{E}$
Elements $(e \in \mathcal{E})$ summarize observables sufficient for diagnosis or replay (e.g., trace fragments, storage diffs, call graphs).
- **Values**: $\mathcal{V}$
Return values of computations.

A single computation step does not merely return success/failure. It returns a **result with evidence**:

$$[R ::= (V \times \mathcal{E}) \cup (F \times \mathcal{E}).]$$

A boundary (b) induces a *recovery obligation*:

- A *recovery procedure family* is a function
 $[\text{Resolve}_b : F \times \mathcal{E} \rightarrow \text{P}(\mathcal{A}_b),]$
 where $(\mathcal{A}_b)$ is a set of admissible recovery actions at boundary (b) , and

$\mathcal{P}$ is powerset.

(We allow nondeterminism here because “recovery options” may be multiple; choosing one can be delegated to policy or a human.)

Recovery closure (informal): boundary b is “closed” when any failure that occurs inside must be transformed into either (i) a resolved continuation producing a value, or (ii) an explicitly exported, classified failure with evidence that is admissible to propagate beyond the boundary.

This is intentionally slightly looser than a full formal closure operator; it is sufficient to support IEEE-style semantic claims and to map cleanly into typed implementations.

IV. The Failure–Recovery Category and Monadic Structure

A. FR morphisms

Fix a boundary $b \in B$. An FR computation from states to states is modeled as a partial step with evidence and classification:

$$[f : X \rightarrow (X \times R).]$$

Read: given current state $x \in X$, either:

- it returns a new state $x' \in X$ and $((v, e) \in V \times E)$, or
- it returns a new state $x' \in X$ and $((\phi, e) \in F \times E)$.

This is the simplest form that simultaneously supports:

- **observability** (evidence always present),
- **resolution** (failures are values in F),
- **localization** (boundary b controls which F is possible and what evidence must contain),
- **composition** (we will define it next).

B. Evidence-carrying Kleisli composition

To compose two computations, we need evidence accumulation. Assume an associative evidence-composition operator:

$$[\oplus : E \times E \rightarrow E]$$

and a neutral element $e_0 \in E$.

$[f : X \rightarrow (X \times \mathbb{R}), \quad g : X \rightarrow (X \times \mathbb{R}),]$
 define $(g \circ_b f)$ by:

1. run $\llbracket f(x) \rrbracket$ producing $\llbracket (x_1, r_1) \rrbracket$,
2. if $\llbracket r_1 \rrbracket$ is success $\llbracket (v_1, e_1) \rrbracket$, then run $\llbracket g(x_1) \rrbracket$ producing $\llbracket (x_2, r_2) \rrbracket$ and combine evidence,
3. if $\llbracket r_1 \rrbracket$ is failure $\llbracket (\phi_1, e_1) \rrbracket$, optionally apply a boundary-specific recovery policy/handler (see §IV-C), otherwise propagate.

Operationally, this is exactly the pattern “failure as data + handlers” familiar from algebraic effects, except we insist evidence is produced regardless of branch.

(homepages.inf.ed.ac.uk (<http://homepages.inf.ed.ac.uk>)) [2])

C. Handlers as recovery obligations

A boundary `(b)` supplies a handler that decides what to do with failures:

$$\backslash [H_b : (F \times E) \rightarrow (X \rightarrow (X \times R)). \backslash$$

Interpretation: given a classified failure plus evidence, the handler produces a continuation computation (possibly requiring human input, i.e., selecting among $\backslash(\mathrm{Resolve}_b(f,e)\backslash)$). This is the formal place where “human annotation” lives: it is not an exception to the theory, but an allowed inhabitant of $\backslash(H_b\backslash)$.

D. Monad laws as recovery invariants

Define the FR endofunctor on computations (informally) by the “result-with-evidence” wrapper, and define:

- **unit** (pure embedding) $\backslash(\eta : X \rightarrow (X \times (V \times E)))\backslash$ with evidence $\backslash(e_0)\backslash$,
- **bind** defined by the composition above plus handler semantics.

Then the **monad laws** become semantic invariants:

$$\backslash(\backslash\mathrm{def}\backslash\mathrm{bind}\{\backslash\mathrm{mathbin}\{>\!>\!=\}\}\backslash)$$

1. Left identity: $\eta(x) \text{ bind } f = f(x)$
2. Right identity: $m \text{ bind } \eta = m$
3. Associativity: $(m \text{ bind } f) \text{ bind } g = m \text{ bind } (\lambda x. f(x) \text{ bind } g)$

In FR terms, these laws assert:

- introducing a “pure step” does not alter recovery behavior,
- eliminating “pure steps” does not alter recovery behavior,

- grouping of recovery-aware compositions does not change meaning.

This matters because recovery is *not* a mere control-flow trick; it is a semantic contract that must remain stable under refactoring and modularization.

V. Seven Implications of Failure-Recovery Theory

This section states implications that follow from the definitions above (with mild assumptions: evidence monoid $(E, \oplus, e_0)$, handler well-typedness, and boundary consistency).

Implication 1: Composability is recovery-preserving morphism existence

Let $\mathcal{C}_b$ be the FR category at boundary b . Two components are composable *in the semantic sense* if the required morphisms exist and compose without violating boundary recovery obligations.

A protocol is composable iff its failure modes admit a handler-complete morphism interface (i.e., failures are classified and resolvable or explicitly exportable with evidence).

This replaces “it usually works” with a checkable semantic criterion.

Implication 2: Safety and composability are orthogonal

Safety is an invariant property (e.g., “no funds are lost”), while composability is the existence of recovery-preserving morphisms.

- A system can be **unsafe yet composable**: failures occur but are classified, observable, and recoverable.
- A system can be **safe yet non-composable**: it is correct in isolation but cannot export failures in a typed, evidence-carrying manner, so integration yields “Unknown” gaps.

This reorders priorities in open systems: non-composability is a first-class failure.

Implication 3: Oracles become “morphism existence” problems, not “truth injection” problems

An oracle interface should not be “inject a value,” but “inject *evidence* that supports a classification and recovery plan.” In FR terms, an oracle is an effect that returns either:

- (v, e) where e certifies provenance, or
- (ϕ, e) where ϕ classifies disagreement/insufficient attestations/etc.

Hence oracle soundness becomes: *does a recovery-preserving morphism exist for oracle disagreement states?*

Implication 4: Human intervention is categorical extension at gaps

A **Gap** is precisely the non-existence of a required morphism in $\mathcal{C}_b$ under the current boundary rules (e.g., “no recovery procedure for this failure class”).

Human annotation is then:

- the act of defining a new handler branch,
- equivalently, extending $\mathcal{C}_b$ by adding a morphism (or refining F / E to make the morphism definable).

This is a formal account of “automation limits”: not mystical, but structural.

Implication 5: “Good” failure growth is recoverable; “bad” failure growth externalizes unrecoverably

Using your terminology mapping:

- **Recoverable Failure Growth:** expanding F while simultaneously expanding H_b and evidence obligations so recovery closure remains attainable.
- **Unrecoverable Externality:** expanding failure surface without providing handlers/evidence, causing failures to leak beyond boundaries without resolution.

Ponzi-style mechanisms are “bad” in FR terms not morally but structurally: they generate failures that are intentionally unrecoverable for victims (no closure), hence externalities are guaranteed.

Implication 6: Strong observability is not optional; it is part of the semantics

Because every result carries evidence, observability is not a debugging afterthought but a semantic requirement. This supports:

- reproducibility,
- auditability,
- counterexample-guided improvement (evidence guides refinement of F and H_b).

This aligns directly with effect handlers as semantic instrumentation rather than runtime tracing hacks. ([homepages.inf.ed.ac.uk (<http://homepages.inf.ed.ac.uk>)] [2])

Implication 7: The EVM forms a privileged experimental substrate for FR

This is the “EVM specialness” claim, now made precise:

EVM provides, at scale, a canonical *sequential* operational semantics with synchronous calls and revert-based atomicity, enabling FR morphisms to be defined over a single, normed execution trace and state transition relation. The Yellow Paper formalizes the baseline semantics and is stable enough to anchor evidence obligations.

([ethereum.github.io (<http://ethereum.github.io>)] [1])

By contrast, systems that (i) bypass consensus for certain transaction classes or (ii) rely on causal ordering for throughput introduce multiple execution regimes; FR can still be done, but the substrate is no longer a single monadic “world trace,” rather a stratified semantics (e.g., owned/shared paths). Sui explicitly adopts this split: many transactions can bypass consensus, while shared-object transactions go through consensus. ([[Sui Documentation](#)] [3])

VI. Theoretical Background of EVM Specialness

This section explains *why* implication 7 holds, as a semantic alignment between FR requirements and EVM properties.

A. Canonical sequential semantics as an evidence normal form

FR needs evidence to be meaningful across composition. EVM’s execution is naturally described as:

- a state transition relation on a global state (accounts/storage),
- a stepwise machine state (stack, memory, program counter),
- a synchronous call stack.

This yields a canonical “trace normal form”: a single linearization (given block order) that is sufficient for replay, debugging, and evidence extraction. The Yellow Paper provides the specification reference point. ([ethereum.github.io (<http://ethereum.github.io>)] [1])

B. Synchronous call semantics matches monadic bind

Your CPS axiom captures the key: synchronous calls pause the caller continuation and resume with return data (or failure). That is exactly the shape needed for straightforward bind:

- caller's continuation = the context held by bind,
- callee result = value passed to the next morphism or failure handled by H_b .

With synchronous calls, “failure as data” is locally compositional.

C. Revert as a built-in recovery primitive (transaction-level atomicity)

While FR does not equate recovery with rollback, EVM's revert/atomicity provides a strong baseline recovery primitive:

- it prevents partial state writes from escaping a failure within a transaction boundary,
- it supports crisp boundary definitions (transaction, call frame, storage region).

This makes “recovery closure” feasible to enforce at well-chosen boundaries.

D. Single-regime execution supports unified theory-to-implementation alignment

Many modern designs split execution into regimes (e.g., bypass consensus vs require consensus). Sui's documentation explicitly distinguishes transactions that may bypass consensus versus those requiring consensus, and highlights causal ordering as a design choice. ([Sui Documentation][3])

This is not “worse,” but it implies FR must become multi-sorted:

- B must include regime tags,
- composability claims become conditional on regime,
- evidence obligations differ across paths.

EVM, in comparison, offers a single dominant regime, making it a cleaner experimental substrate for first-principles FR work.

VII. Engineering Interpretation: From Theory to Typed Systems (Idris2-facing)

Although this article is not an implementation report, it is designed to map cleanly to typed languages (e.g., Idris2) and to EVM-targeted DSLs:

- F becomes a closed sum type `Failure` .
- E becomes a structured evidence record `Evidence` .
- R becomes `Either (Failure, Evidence) (Value, Evidence)` .
- H_b becomes a handler stack whose type enforces:
 1. failure classification,
 2. evidence production,
 3. resolution availability (possibly via “human annotation” token).

Boundaries $b \in B$ naturally correspond to indexed-monad parameters (capabilities, epochs, entry context), ensuring “localize/observe/classify/resolve” is enforced by type construction rather than convention.

VIII. Relation to Prior Work

The FR theory can be read as an application-specialized instance of:

- algebraic effects and handlers, where failure and recovery are first-class effects with handlers, and evidence is a mandatory output component.
([homepages.inf.ed.ac.uk (<http://homepages.inf.ed.ac.uk>)] [2])
- formal VM specifications, with EVM as a prominent reference model.
([ethereum.github.io (<http://ethereum.github.io>)] [1])
- execution-regime designs in newer chains (e.g., causal ordering / consensus bypass for subsets of transactions). ([Sui Documentation] [3])

IX. Conclusion

We presented a Failure–Recovery theory for world-computer VMs by:

1. defining failure surface, evidence, and recovery closure,
2. constructing a recovery-aware evidence-carrying morphism semantics,
3. interpreting monad laws as invariants of recovery stability under composition,
4. deriving seven implications that reframe composability, oracle design, and human intervention, and
5. explaining why the EVM is a privileged experimental substrate for FR.

The key conceptual payoff is a shift from “systems that avoid failure” to **systems that make failure classifiable, observable, localized, and recoverable under composition.**

References (informal list for manuscript drafting)

- G. Wood, *Ethereum: A Secure Decentralised Generalised Transaction Ledger (Yellow Paper)* (Shanghai version). ([ethereum.github.io (<http://ethereum.github.io>)] [1])
- G. Plotkin and M. Pretnar, *Handlers of Algebraic Effects* (2009) and related treatments. ([homepages.inf.ed.ac.uk (<http://homepages.inf.ed.ac.uk>)] [2])
- M. Pretnar, *An Introduction to Algebraic Effects and Handlers* (tutorial). ([ScienceDirect] [4])
- Sui documentation on causal ordering and consensus bypass / shared objects. ([Sui Documentation] [3])
- IEEE Author Center guidance for abstracts/keywords (for formatting discipline). ([IEEE Author Center Journals] [5])